Annotation Guideline for Temporal Code-Comment Drift

Supplementary Material for the Submitted Manuscript.

1. Core Principle

The sole criterion for annotation is: *Does the semantic meaning, behavior, or interface contract described by the historical comment still accurately reflect the current code?* We evaluate the external behavior of the code, not its internal implementation details (e.g., variable renamings).

2. Definition of DRIFT (Inconsistent)

Annotate as **DRIFT** if the current code meets *any* of the following conditions:

- **Logic Reversal/Change:** The code performs the opposite operation or the core algorithm has been replaced.
- **Interface Contract Change:** Parameter types, return data structures, or special return values (e.g., returns `null` vs. an empty list) have changed.
- **Exception Behavior:** Declared exceptions in the comment are removed, or fatal undocumented exceptions are introduced.
- **Boundary/Condition Change:** Validation logic (e.g., `if` conditions) has changed, altering the triggering scenario.
- **Invisible Side Effects:** The code adds side effects (e.g., database writes, network requests) not mentioned in the comment.

3. Definition of NO_DRIFT (Consistent/Refactoring)

Annotate as **NO_DRIFT** if the changes are strictly internal and preserve semantics:

- **Equivalent Refactoring:** Renaming local variables, replacing `for` loops with Stream/Lambda, or extracting code into private methods.
- **Non-destructive Enhancements:** Adding logging, assertions, or defensive null-checks (`if (obj == null)`) that do not alter normal input logic.
- **Ancillary Modifications:** Changing error messages (without changing exception types), adjusting code formatting, or changing access modifiers (e.g., `public` to `private`).

4. Definition of SKIP (Ambiguous)

Skip if the comment is extremely vague (e.g., `// Handles data`), if the code snippet relies heavily on missing external context, or if the annotator is uncertain. Ambiguous samples are excluded from the benchmark.